

Aula 1 (Módulo 06) — Fundamentos de Python: variáveis, tipos, condições, loops, funções

1. Por que Python para Blue Team

Python é a linguagem mais usada em automação de segurança porque é **fácil de ler**, tem uma biblioteca padrão enorme (lidar com arquivos, texto, rede, data/hora já vem pronto) e roda em qualquer sistema operacional que já estudamos (Linux, Windows, macOS). Um analista que sabe programar consegue automatizar tarefas repetitivas — em vez de checar 500 logs manualmente, escreve um script que faz isso em segundos.

2. Rodando Python

```
python3 --version
python3
```

O segundo comando abre o **interpretador interativo** — você digita uma linha de código, aperta Enter, e vê o resultado na hora. Ótimo para testar coisas rápidas. Para sair, digite `exit()` ou `Ctrl+D`.

Para scripts de verdade, criamos um arquivo `.py` e rodamos com:

```
python3 meu_script.py
```

3. Variáveis e tipos

Uma **variável** guarda um valor com um nome que você escolhe:

```
usuario = "ana"
tentativas_falhas = 5
ip_suspeito = "203.0.113.45"
esta_bloqueado = False
```

Os **tipos** básicos que você vai usar o tempo todo:

Tipo	Exemplo	Uso típico em Blue Team
<code>str</code> (texto)	<code>"203.0.113.45"</code>	IPs, nomes de usuário, linhas de log
<code>int</code> (número inteiro)	<code>5</code>	contagens, PIDs, portas
<code>float</code> (número decimal)	<code>12.5</code>	tempos, percentuais
<code>bool</code> (verdadeiro/falso)	<code>True</code> , <code>False</code>	"esse IP está na blocklist?"
<code>list</code> (lista, ordenada)	<code>["ana", "root", "admin"]</code>	lista de usuários vistos em um log
<code>dict</code> (dicionário, chave → valor)	<code>{"ip": "203.0.113.45", "tentativas": 5}</code>	um evento de log estruturado

```
# listas: acessadas por posição (começando em 0)
usuarios = ["ana", "root", "admin"]
print(usuarios[0])      # ana
print(len(usuarios))    # 3 (quantidade de itens)

# dicionários: acessados por chave
evento = {"ip": "203.0.113.45", "tentativas": 5, "usuario": "root"}
print(evento["ip"])     # 203.0.113.45
print(evento["tentativas"]) # 5
```

4. Condições (`if`)

Permite que o código **decida** o que fazer com base em uma condição — exatamente a lógica por trás de qualquer regra de detecção (lembra da regra Suricata do Módulo 04, Aula 2? "SE mais de 5 tentativas EM 60 segundos, ENTÃO alerta").

```
tentativas_falhas = 7

if tentativas_falhas > 5:
    print("Possível brute force!")
elif tentativas_falhas > 0:
    print("Algumas falhas, mas dentro do normal.")
else:
    print("Nenhuma falha.")
```

Operadores de comparação: `>`, `<`, `>=`, `<=`, `==` (igual), `!=` (diferente). Note: **atenção** — `=` atribui um valor (`x = 5`), `==` compara (`x == 5`). Confundir os dois é um erro comum de iniciante.

5. Loops (`for` e `while`)

Repetem uma ação várias vezes — essencial para processar **muitas** linhas de log de uma vez, em vez de uma por uma manualmente.

```
ips_suspeitos = ["203.0.113.45", "198.51.100.10", "192.0.2.77"]

for ip in ips_suspeitos:
    print(f"Verificando o IP: {ip}")
```

(O `f"texto {variavel}"` se chama **f-string** — insere o valor de uma variável dentro de um texto. Muito usado para montar mensagens/logs.)

```
tentativas = 0
while tentativas < 3:
    print(f"Tentativa número {tentativas}")
    tentativas = tentativas + 1    # ou: tentativas += 1
```

`while` repete **enquanto** a condição for verdadeira — cuidado para sempre ter algo que eventualmente torne a condição falsa, senão o loop nunca para (chamado de *loop infinito*).

6. Funções

Uma **função** empacota um pedaço de código reutilizável, com um nome, que pode receber entradas (**parâmetros**) e devolver um resultado (**return**).

```
def eh_ip_privado(ip):
    """Verifica se um IP começa com um prefixo de faixa privada (RFC 1918)."""
    return ip.startswith("10.") or ip.startswith("192.168.") or ip.startswith("172.16.")

resultado = eh_ip_privado("192.168.1.10")
print(resultado)    # True

resultado2 = eh_ip_privado("8.8.8.8")
print(resultado2)   # False
```

Repare que `eh_ip_privado` usa exatamente o conceito de IP privado que vimos no Módulo 03, Aula 1 — Python é onde a teoria de rede que já estudamos vira uma ferramenta de verdade.

7. Juntando tudo: um mini-exemplo de triagem

```
def classificar_tentativas(tentativas_falhas):
    if tentativas_falhas > 10:
        return "CRÍTICO"
    elif tentativas_falhas > 5:
        return "ALTO"
    elif tentativas_falhas > 0:
        return "BAIXO"
    else:
        return "NORMAL"

eventos = [
    {"ip": "203.0.113.45", "tentativas": 12},
    {"ip": "198.51.100.10", "tentativas": 3},
    {"ip": "192.0.2.77", "tentativas": 0},
]

for evento in eventos:
    severidade = classificar_tentativas(evento["tentativas"])
    print(f"IP {evento['ip']}: severidade {severidade}")
```

Isso já é, em miniatura, a mesma lógica de um **motor de correlação de SIEM** (Módulo 09) — uma regra decidindo severidade com base em dados estruturados.

8. Exercício prático

Crie um arquivo `pratica_aula1.py` (pode ser em qualquer pasta local sua de estudo, ou dentro de `scripts/` deste repositório) com o seguinte desafio: dada a lista de eventos abaixo, imprima **só** os IPs que tiveram mais de 5 tentativas falhas, usando `for` + `if`:

```
eventos = [
    {"ip": "203.0.113.45", "tentativas": 12},
    {"ip": "198.51.100.10", "tentativas": 3},
    {"ip": "192.0.2.77", "tentativas": 8},
]

# seu código aqui: imprimir só os IPs com tentativas > 5
```

(Resultado esperado: `203.0.113.45` e `192.0.2.77`.)

9. Recapitulando

- Variáveis guardam valores; tipos principais: `str`, `int`, `float`, `bool`, `list`, `dict`.
- `if` / `elif` / `else` decide com base em condições — a lógica por trás de qualquer regra de detecção.
- `for` repete sobre uma lista; `while` repete enquanto uma condição for verdadeira.

- Funções (`def`) empacotam lógica reutilizável, com parâmetros e `return`.

Próxima aula

Arquivos, regex, JSON e CSV — como ler logs de verdade, extrair padrões (como IPs e IOCs) com expressões regulares, e trabalhar com os formatos de dados mais comuns em segurança.

Fontes recomendadas

- **Python — documentação oficial ("The Python Tutorial")**: referência oficial e completa da linguagem.
<https://docs.python.org/3/tutorial/>
- **Python — "Built-in Types"**: referência oficial de todos os tipos nativos (`str`, `list`, `dict`, etc.).
<https://docs.python.org/3/library/stdtypes.html>